
DEVIXO SOLUTIONS

AI/ML Internship Program

Task 04: End-to-End Machine Learning Project

House Price Prediction System

Student Name:

Maryam Khan

Internship Domain:

Artificial Intelligence / Machine Learning

Submission Date:

13/09/2026

1. Introduction

Machine learning has become a central tool for solving real-world prediction problems by learning patterns from historical data. This report documents the design, development, and deployment of a complete, end-to-end machine learning application built as Task 4 of the AI/ML Internship Program at Devixo Solutions. Rather than stopping at a Jupyter Notebook analysis, this project follows the full lifecycle of a production machine learning system: acquiring and cleaning real-world data, engineering meaningful features, training and comparing multiple models, tuning the best performer, and finally packaging the solution into an interactive, publicly deployed web application.

The chosen problem is house price prediction — a well-known and practically relevant regression task that allows for rich feature engineering, meaningful model comparison, and an intuitive, easy-to-demonstrate user interface.

2. Objective

The objective of this task was to build a complete machine learning application that takes real-world user input, processes it, generates a prediction, and presents the result through an interactive interface. Specifically, the project aimed to:

- Load and clean a real-world, publicly available dataset.
- Apply thorough data preprocessing, including handling of missing values, duplicates, outliers, categorical encoding, feature scaling, and feature engineering.
- Train and compare at least four machine learning models using appropriate regression metrics.
- Apply hyperparameter tuning to the best-performing model and quantify the improvement.
- Save the final trained model for reuse in a production setting.
- Build a professional, interactive Streamlit web application allowing users to enter house details and receive a price prediction.
- Deploy the application publicly so it can be accessed and used online.

3. Dataset Name

House Prices — Advanced Regression Techniques (commonly known as the Ames Housing Dataset).

4. Dataset Source

The dataset was sourced from Kaggle's "House Prices - Advanced Regression Techniques" competition, which is itself compiled from residential property sale records for Ames, Iowa, USA.

<https://www.kaggle.com/c/house-prices-advanced-regression-techniques/data>

5. Dataset Description

The dataset contains detailed records of residential home sales, describing nearly every aspect of a property that could influence its sale price — from lot size and location to construction quality, interior

finish, and the presence of amenities such as garages, basements, and fireplaces. It combines numerical features (e.g., square footage, year built) with categorical features (e.g., neighborhood, roofing material, sale condition), making it an excellent dataset for demonstrating a realistic, comprehensive data preprocessing pipeline. The target variable to be predicted is SalePrice, the final sale price of each home in US dollars.

6. Dataset Size

Stage	Rows	Columns
Original training data	1,460	81 (80 features + SalePrice)
After duplicate check	1,460	80 (Id column dropped)
After outlier removal	1,458	80
After encoding & feature engineering	1,458	240 (239 features + SalePrice)

7. Columns Description

The full dataset contains 80 original feature columns, described in detail in the accompanying data_description.txt file. A representative selection of the most influential columns is summarized below:

Column	Description
OverallQual	Overall material and finish quality, rated 1 (Poor) to 10 (Excellent)
GrLivArea	Above-ground living area, in square feet
TotalBsmtSF	Total basement area, in square feet
GarageArea / GarageCars	Garage size in square feet / car capacity
YearBuilt / YearRemodAdd	Year the house was built / last remodeled
Neighborhood	Physical location within Ames city limits
KitchenQual	Kitchen quality (Excellent, Good, Average, Fair, Poor)
FullBath / HalfBath	Number of full and half bathrooms above grade
SalePrice	Target variable — final sale price of the home in USD

8. Project Structure

```
house-price-prediction-task4/
├── app/
│   ├── app.py          # Streamlit application
│   └── requirements.txt # Dependencies (used by Streamlit Cloud deployment)
├── data/
│   ├── train.csv
│   ├── test.csv
│   ├── data_description.txt
│   └── processed_train.csv
├── models/
│   ├── final_model.pkl # Tuned Gradient Boosting model (final)
│   ├── all_models.pkl  # All 4 trained models (for model selection feature)
│   ├── scaler.pkl
│   ├── cols_to_scale.pkl
│   ├── feature_columns.pkl
│   └── feature_defaults.pkl
├── notebooks/
│   └── Task_04_ML.ipynb # Full data processing & model development notebook
├── report/
│   └── Task4_Report.pdf
├── generate_defaults.py # Generates default feature values for the app's input form
├── retrain_local.py    # Retrains and saves the final model locally
├── save_all_models.py  # Trains and saves all 4 models for model selection
├── .gitignore
└── README.md
```

9. Tools & Technologies Used

Category	Tools / Libraries
Language	Python 3
Data Processing	Pandas, NumPy
Machine Learning	Scikit-learn (Linear Regression, Ridge, Random Forest, Gradient Boosting)
Visualization	Matplotlib
Web Application	Streamlit
Model Persistence	Joblib
Version Control	Git, GitHub
Deployment	Streamlit Community Cloud
Development Environment	Google Colab (analysis), Visual Studio Code (application development)

10. Data Loading

The dataset was loaded using

```
df = pd.read_csv("data/train.csv")
```

```
df = pd.read_csv("data/train.csv")
print(f"Loaded dataset: {df.shape[0]} rows, {df.shape[1]} columns")
df = df.drop(columns=["Id"])
df.head()
```

... Loaded dataset: 1460 rows, 81 columns

	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	...	MiscFeature	MiscVal	MoSold	YrSold	SaleType	SaleCondition	Sa
0	60	RL	65.0	8450	Pave	NaN	Reg	...	NaN	0	2	2008	WD	Normal	
1	20	RL	80.0	9600	Pave	NaN	Reg	...	NaN	0	5	2007	WD	Normal	
2	60	RL	68.0	11250	Pave	NaN	IR1	...	NaN	0	9	2008	WD	Normal	
3	70	RL	60.0	9550	Pave	NaN	IR1	...	NaN	0	2	2006	WD	Abnorml	
4	60	RL	84.0	14260	Pave	NaN	IR1	...	NaN	0	12	2008	WD	Normal	

5 rows x 80 columns

11. Data Preprocessing

11.1 Handling Missing Values

A total of 7,829 missing values were identified across 19 columns. Rather than applying a single blanket strategy, missing values were handled based on their actual meaning:

- Columns such as PoolQC, Alley, FireplaceQu, and similar features where a missing value genuinely means “this feature does not exist” (e.g., no pool, no alley access) were filled with the category “None”.
- Related numeric columns (e.g., GarageYrBlt, MasVnrArea) were filled with 0 to match the absence of that feature.
- LotFrontage, a genuinely missing numeric value, was imputed using the median LotFrontage within the same Neighborhood, since lot size correlates strongly with location.
- Electrical (a single missing value) was filled using the column mode.

After this process, all 7,829 missing values were fully resolved with zero remaining nulls.

```
missing_before = df.isnull().sum().sum()
print(f"Total missing values before cleaning: {missing_before}")

... Total missing values before cleaning: 7829
```

```
# Group A: columns where NaN genuinely means "feature doesn't exist"
none_means_no_feature = [
    "PoolQC", "MiscFeature", "Alley", "Fence", "FireplaceQu",
    "GarageType", "GarageFinish", "GarageQual", "GarageCond",
    "BsmtQual", "BsmtCond", "BsmtExposure", "BsmtFinType1", "BsmtFinType2",
    "MasVnrType"
]
for col in none_means_no_feature:
    df[col] = df[col].fillna("None")

# Group B: numeric columns tied to the "no feature" columns above
df["GarageYrBlt"] = df["GarageYrBlt"].fillna(0)
df["MasVnrArea"] = df["MasVnrArea"].fillna(0)

# Group C: LotFrontage - genuinely missing numeric -> fill with median per Neighborhood
df["LotFrontage"] = df.groupby("Neighborhood")["LotFrontage"].transform(
    lambda x: x.fillna(x.median())
)

# Group D: Electrical - only 1 missing, categorical -> use mode
df["Electrical"] = df["Electrical"].fillna(df["Electrical"].mode()[0])
```

```

missing_after = df.isnull().sum().sum()
print(f"Total missing values after cleaning: {missing_after}")
assert missing_after == 0, "There are still missing values left!"
print("All missing values resolved.")

... Total missing values after cleaning: 0
All missing values resolved.

print(df.isnull().sum().sum())
print(df.isnull().sum().sort_values(ascending=False).head(10))

... 0
MSSubClass      0
MSZoning        0
LotFrontage     0
LotArea         0
Street          0
Alley           0
LotShape        0
LandContour     0
Utilities       0
LotConfig       0
dtype: int64

```

11.2 Removing Duplicates

The dataset was checked using `drop_duplicates()` and confirmed to contain zero duplicate rows, indicating a clean source dataset.

```

before = df.shape[0]
df = df.drop_duplicates()
after = df.shape[0]
print(f"Rows before: {before} | Rows after: {after} | Duplicates removed: {before - after}")

... Rows before: 1460 | Rows after: 1460 | Duplicates removed: 0

```

11.3 Outlier Detection

An IQR-based statistical check flagged 61 potential outliers in the `SalePrice` distribution; these were retained, as they represent genuinely expensive homes rather than data errors. However, two well-documented anomalies were removed: houses with unusually large living area (greater than 4,000 square feet) sold at unusually low prices (below \$300,000) — a known data quirk in this dataset, confirmed by visual inspection of a `GrLivArea` vs. `SalePrice` scatter plot.

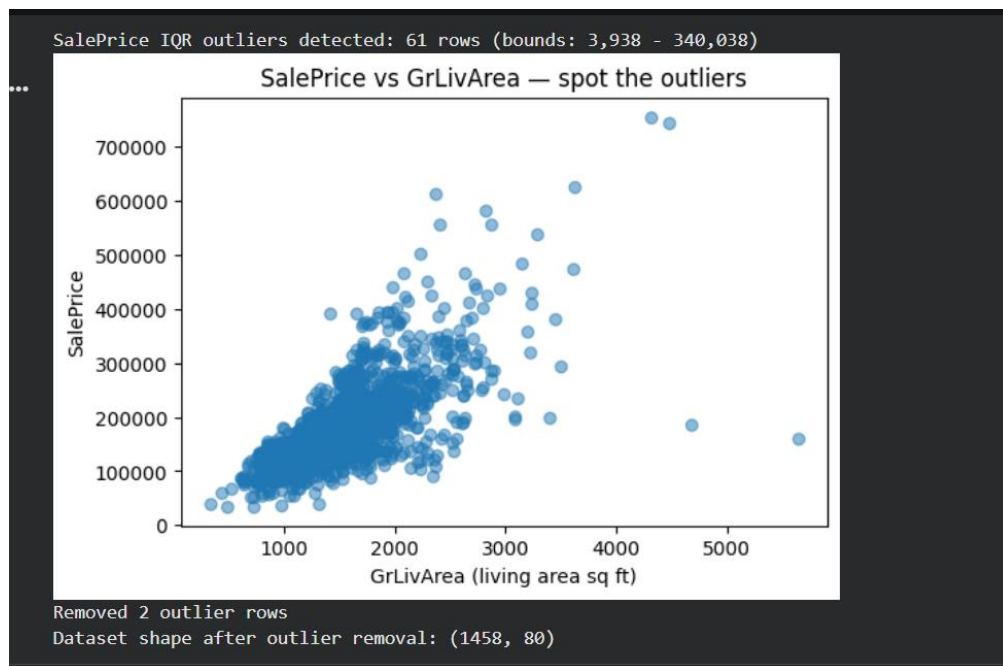
```

# IQR-based outlier detection on the target variable
Q1 = df["SalePrice"].quantile(0.25)
Q3 = df["SalePrice"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
price_outliers = df[(df["SalePrice"] < lower_bound) | (df["SalePrice"] > upper_bound)]
print(f"SalePrice IQR outliers detected: {len(price_outliers)} rows "
      f"(bounds: {lower_bound:,.0f} - {upper_bound:,.0f})")

# Visualize a well-known outlier pattern in this dataset
import matplotlib.pyplot as plt
plt.figure(figsize=(6,4))
plt.scatter(df["GrLivArea"], df["SalePrice"], alpha=0.5)
plt.xlabel("GrLivArea (living area sq ft)")
plt.ylabel("SalePrice")
plt.title("SalePrice vs GrLivArea - spot the outliers")
plt.show()

# Remove well-documented extreme outliers: very large houses (>4000 sqft)
before_outlier_removal = df.shape[0]
df = df[~((df["GrLivArea"] > 4000) & (df["SalePrice"] < 300000))]
after_outlier_removal = df.shape[0]
print(f"Removed {before_outlier_removal - after_outlier_removal} outlier rows")
print(f"Dataset shape after outlier removal: {df.shape}")

```



11.4 Encoding Categorical Variables

Ten quality/condition columns with a natural order (e.g., KitchenQual, ExterQual) were ordinal-encoded on a 0–5 scale (Excellent = 5 ... Poor = 1, None = 0). The remaining 33 nominal categorical columns (e.g., Neighborhood, RoofStyle) were one-hot encoded using `pd.get_dummies()`, expanding the dataset from 80 to 229 columns.

```
# Ordinal encoding for quality/condition scales
quality_map = {"Ex": 5, "Gd": 4, "TA": 3, "Fa": 2, "Po": 1, "None": 0}
ordinal_cols = ["ExterQual", "ExterCond", "BsmtQual", "BsmtCond", "HeatingQC",
               "KitchenQual", "FireplaceQu", "GarageQual", "GarageCond", "PoolQC"]

for col in ordinal_cols:
    df[col] = df[col].map(quality_map)

print(f"Ordinal-encoded {len(ordinal_cols)} quality/condition columns")

# One-hot encoding for remaining nominal categorical columns
nominal_cols = df.select_dtypes(include="object").columns.tolist()
print(f"One-hot encoding {len(nominal_cols)} nominal columns: {nominal_cols}")

df = pd.get_dummies(df, columns=nominal_cols, drop_first=True)
print(f"Shape after encoding: {df.shape}")

*** Ordinal-encoded 10 quality/condition columns
One-hot encoding 33 nominal columns: ['MSZoning', 'Street', 'Alley', 'LotShape', 'LandContour', 'Utilities', 'LotConfig', 'LandSlope', 'Neigh
Shape after encoding: (1458, 229)
```

11.5 Feature Engineering

Eleven new features were engineered from existing columns to better capture patterns relevant to price:

- TotalSF — combined basement, first floor, and second floor square footage.
- HouseAge and RemodAge — years since construction / remodeling, relative to the year of sale.
- TotalBath — a single weighted count of all bathrooms (half-baths counted as 0.5).
- TotalPorchSF — combined porch area across all porch types.
- Binary indicator flags: HasPool, HasGarage, HasFireplace, Has2ndFloor, HasBasement, IsRemodeled.

```

# Feature engineering – new features derived from existing raw columns
df["TotalSF"] = df["TotalBsmstSF"] + df["1stFlrSF"] + df["2ndFlrSF"]
df["HouseAge"] = df["YrSold"] - df["YearBuilt"]
df["RemodAge"] = df["YrSold"] - df["YearRemodAdd"]
df["TotalBath"] = (df["FullBath"] + 0.5 * df["HalfBath"] +
                  df["BsmstFullBath"] + 0.5 * df["BsmstHalfBath"])
df["TotalPorchSF"] = (df["OpenPorchSF"] + df["EnclosedPorch"] +
                     df["3SsnPorch"] + df["ScreenPorch"] + df["WoodDeckSF"])

df["HasPool"] = (df["PoolArea"] > 0).astype(int)
df["HasGarage"] = (df["GarageArea"] > 0).astype(int)
df["HasFireplace"] = (df["Fireplaces"] > 0).astype(int)
df["Has2ndFloor"] = (df["2ndFlrSF"] > 0).astype(int)
df["HasBasement"] = (df["TotalBsmstSF"] > 0).astype(int)
df["IsRemodeled"] = (df["YearBuilt"] != df["YearRemodAdd"]).astype(int)

new_features = ["TotalSF", "HouseAge", "RemodAge", "TotalBath", "TotalPorchSF",
               "HasPool", "HasGarage", "HasFireplace", "Has2ndFloor",
               "HasBasement", "IsRemodeled"]
print(f"Created {len(new_features)} new features")
print(f"Shape after feature engineering: {df.shape}")

... Created 11 new features
Shape after feature engineering: (1458, 240)

```

11.6 Feature Scaling

StandardScaler was applied only to the 51 genuinely continuous numeric columns. Binary flags and one-hot encoded dummy columns (188 in total) were deliberately left unscaled, as scaling 0/1 values provides no benefit and reduces interpretability. The final processed dataset contains 1,458 rows and 240 columns (239 features plus the target).

```

# Separate features (X) from target (y) – never scale the target itself
X = df.drop(columns=["SalePrice"])
y = df["SalePrice"]

# Only scale genuinely continuous columns – not binary flags or one-hot dummies
binary_like_cols = [c for c in X.columns if X[c].dropna().isin([0, 1]).all()]
cols_to_scale = [c for c in X.select_dtypes(include=[np.number]).columns
                 if c not in binary_like_cols]

scaler = StandardScaler()
X[cols_to_scale] = scaler.fit_transform(X[cols_to_scale])

print(f"Scaled {len(cols_to_scale)} continuous columns")
print(f"Left {len(binary_like_cols)} binary/dummy columns unscaled")

# Save the scaler for later use in the Streamlit app
joblib.dump(scaler, "scaler.pkl")
joblib.dump(cols_to_scale, "cols_to_scale.pkl")
joblib.dump(X.columns.tolist(), "feature_columns.pkl")
print("Saved scaler.pkl, cols_to_scale.pkl, feature_columns.pkl")

# Recombine into final processed dataset
final = X.copy()
final["SalePrice"] = y.values
final.to_csv("processed_train.csv", index=False)
print(f"Saved processed_train.csv -> shape: {final.shape}")

... Scaled 51 continuous columns
Left 188 binary/dummy columns unscaled
Saved scaler.pkl, cols_to_scale.pkl, feature_columns.pkl
Saved processed_train.csv -> shape: (1458, 240)

```

12. Model Development

Four regression models were trained on an 80/20 train-test split (1,166 training rows, 292 test rows):

- Linear Regression — a simple baseline assuming linear relationships between features and price.
- Ridge Regression — a linear model with L2 regularization, better suited to handle multicollinearity across 239 features.
- Random Forest Regressor — an ensemble of decision trees, capable of capturing non-linear feature interactions.
- Gradient Boosting Regressor — an ensemble method that builds trees sequentially, each correcting the errors of the last.

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
print(f"Train set: {X_train.shape}")  
print(f"Test set: {X_test.shape}")
```

```
Train set: (1166, 239)  
Test set: (292, 239)
```

```
from sklearn.linear_model import LinearRegression, Ridge  
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor  
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
import numpy as np  
import pandas as pd  
  
models = {  
    "Linear Regression": LinearRegression(),  
    "Ridge Regression": Ridge(alpha=1.0, random_state=42),  
    "Random Forest": RandomForestRegressor(n_estimators=200, random_state=42, n_jobs=-1),  
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=200, random_state=42),  
}  
  
results = []  
trained_models = {}  
  
for name, model in models.items():  
    model.fit(X_train, y_train)  
    preds = model.predict(X_test)  
  
    mae = mean_absolute_error(y_test, preds)  
    mse = mean_squared_error(y_test, preds)  
    rmse = np.sqrt(mse)  
    r2 = r2_score(y_test, preds)
```

```
results.append({"Model": name, "MAE": mae, "MSE": mse, "RMSE": rmse, "R2 Score": r2})  
trained_models[name] = model  
print(f"{name} trained.")
```

```
results_df = pd.DataFrame(results).sort_values("RMSE").reset_index(drop=True)  
results_df
```

```
... Linear Regression trained.  
Ridge Regression trained.  
Random Forest trained.  
Gradient Boosting trained.
```

	Model	MAE	MSE	RMSE	R2 Score
0	Gradient Boosting	14571.031900	4.169056e+08	20418.266199	0.924525
1	Random Forest	16133.700634	5.716714e+08	23909.651667	0.896506
2	Ridge Regression	18811.672263	6.517627e+08	25529.643476	0.882007
3	Linear Regression	19588.490082	7.540374e+08	27459.741668	0.863491

13. Model Evaluation

Since this is a regression problem, models were evaluated using four standard regression metrics:

- MAE (Mean Absolute Error) — the average absolute difference between predicted and actual prices.
- MSE (Mean Squared Error) — the average squared difference, penalizing larger errors more heavily.
- RMSE (Root Mean Squared Error) — the square root of MSE, expressed in the same units as the target (dollars).
- R^2 Score — the proportion of variance in SalePrice explained by the model.

14. Model Comparison

Model	MAE (\$)	RMSE (\$)	R^2 Score
Gradient Boosting	14,571	20,418	0.9245
Random Forest	16,134	23,910	0.8965
Ridge Regression	18,812	25,530	0.8820
Linear Regression	19,588	27,460	0.8635

Gradient Boosting achieved the lowest error and highest R^2 score, and was therefore selected as the base model for hyperparameter tuning. This result aligns with expectations: tree-based ensemble methods typically outperform linear models on tabular data with complex, non-linear feature interactions, as is the case here.

15. Hyperparameter Tuning

RandomizedSearchCV was applied to the Gradient Boosting model, testing 30 random combinations across a defined parameter grid, each evaluated using 5-fold cross-validation — a total of 150 model fits.

The best parameters identified were: {'subsample': 0.8, 'n_estimators': 400, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_depth': 2, 'learning_rate': 0.1}

```

from sklearn.model_selection import RandomizedSearchCV
param_dist = {
    "n_estimators": [100, 200, 300, 400],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "max_depth": [2, 3, 4, 5],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4],
    "subsample": [0.7, 0.8, 0.9, 1.0],
}
random_search = RandomizedSearchCV(
    estimator=GradientBoostingRegressor(random_state=42),
    param_distributions=param_dist,
    n_iter=30,
    scoring="neg_root_mean_squared_error",
    cv=5,
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search.fit(X_train, y_train)
print("Best parameters found:")
print(random_search.best_params_)

... Fitting 5 folds for each of 30 candidates, totalling 150 fits
Best parameters found:
{'subsample': 0.8, 'n_estimators': 400, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_depth': 2, 'learning_rate': 0.1}

```

Performance Before vs. After Tuning

Metric	Before Tuning	After Tuning	Improvement
MAE	14,571	13,887	− 684
MSE	416,905,600	401,826.900	− 15,078, 700
RMSE	20,418	20,046	− 372
R ² Score	0.9245	0.9273	+ 0.0027

Tuning produced a consistent, modest improvement across every metric — evidence that the tuning process genuinely optimized the model rather than the untuned model already being at its performance ceiling by chance.

```

# Get the tuned model
tuned_model = random_search.best_estimator_

# Evaluate tuned model on the test set
tuned_preds = tuned_model.predict(X_test)

tuned_mae = mean_absolute_error(y_test, tuned_preds)
tuned_mse = mean_squared_error(y_test, tuned_preds)
tuned_rmse = np.sqrt(tuned_mse)
tuned_r2 = r2_score(y_test, tuned_preds)

# Original (untuned) Gradient Boosting metrics, for comparison
original_metrics = results_df[results_df["Model"] == "Gradient Boosting"].iloc[0]

comparison_df = pd.DataFrame({
    "Metric": ["MAE", "MSE", "RMSE", "R2 Score"],
    "Before Tuning": [original_metrics["MAE"], original_metrics["MSE"],
        original_metrics["RMSE"], original_metrics["R2 Score"]],
    "After Tuning": [tuned_mae, tuned_mse, tuned_rmse, tuned_r2]
})

comparison_df["Improvement"] = comparison_df["Before Tuning"] - comparison_df["After Tuning"]
comparison_df.loc[comparison_df["Metric"] == "R2 Score", "Improvement"] = \
    tuned_r2 - original_metrics["R2 Score"]
print(comparison_df)

```

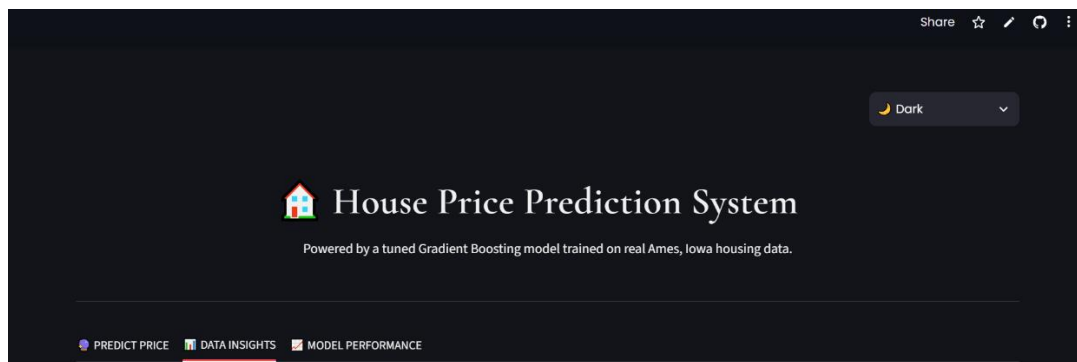
...	Metric	Before Tuning	After Tuning	Improvement
0	MAE	1.457103e+04	1.388762e+04	6.834142e+02
1	MSE	4.169056e+08	4.018269e+08	1.507869e+07
2	RMSE	2.041827e+04	2.004562e+04	3.726455e+02
3	R2 Score	9.245246e-01	9.272544e-01	2.729803e-03

16. Model Deployment

The final tuned Gradient Boosting model, along with the fitted StandardScaler and supporting metadata (feature column order, scaling column list, and default feature values), was serialized using joblib. All four trained models were also saved together to support the application's multiple model selection feature. The application and its dependencies were pushed to a public GitHub repository and deployed live using Streamlit Community Cloud, which automatically installs dependencies from requirements.txt and redeploys on every new commit to the main branch.

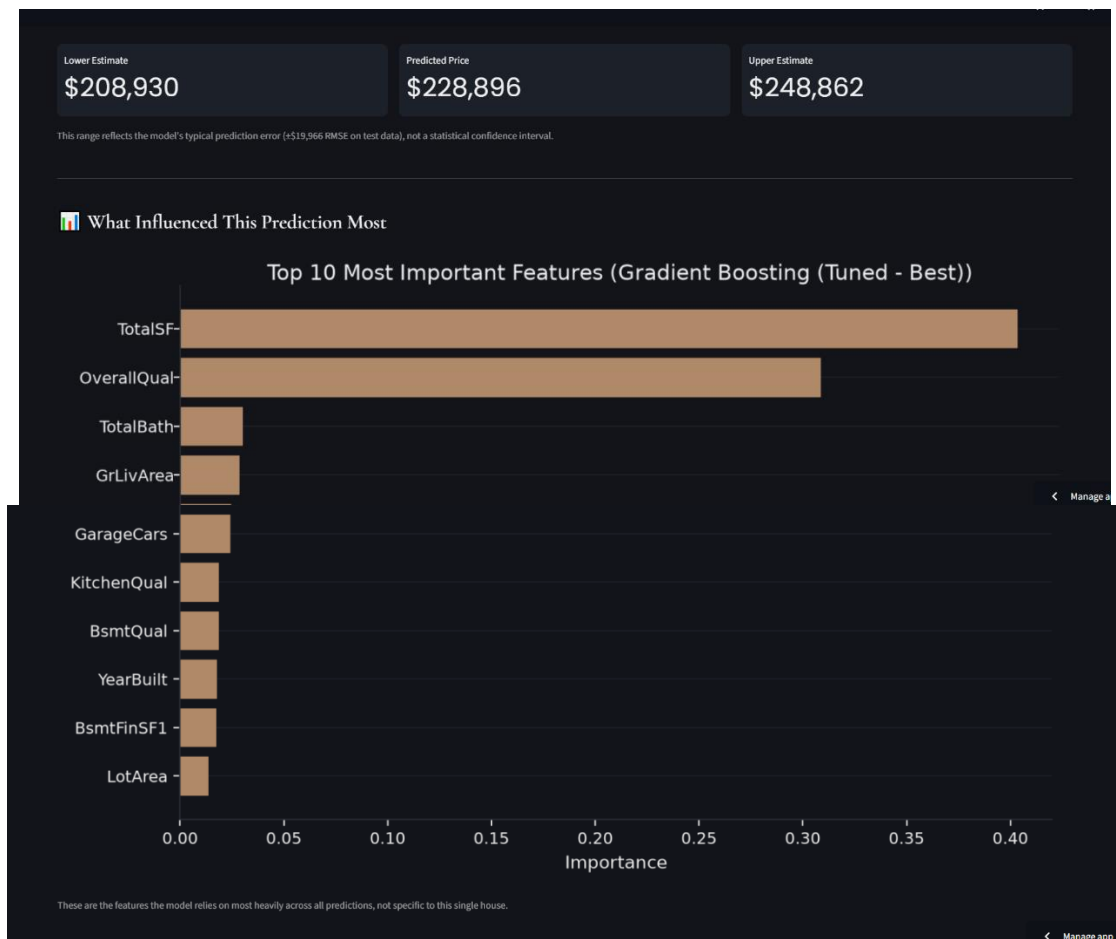
17. Web Application

A multi-tab Streamlit web application was built to provide a professional, interactive interface for the trained model. The application includes three main sections, accessible via a styled navigation bar, along with a Light/Dark appearance toggle.



Predict Price

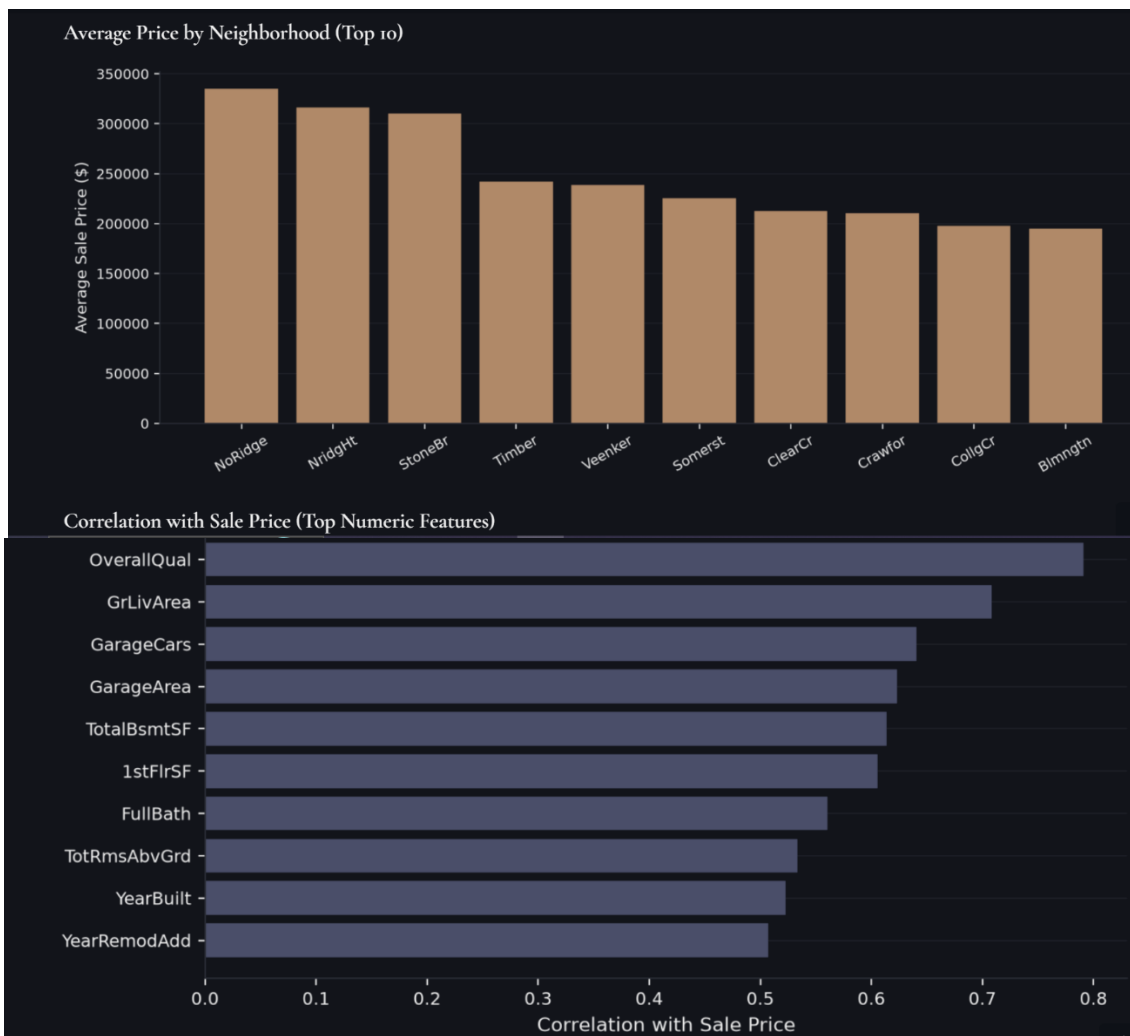
Users enter house details across three grouped sections (Size, Rooms & Quality, Location & Age), select which trained model should generate the prediction, and click “Predict Price” to receive an estimated sale price.



Data Insights

A dedicated visualization page presenting the sale price distribution, price versus living area relationship, average price by neighborhood, and correlation of numeric features with sale price.





Model Performance

A page comparing all four trained models side by side, alongside a before-and-after hyperparameter tuning comparison table.

 Model Performance				
Comparison of all models trained during development.				
	Model	MAE	RMSE	R2 Score
0	Gradient Boosting	14571	20418	0.9245
1	Random Forest	16134	23910	0.8965
2	Ridge Regression	18812	25530	0.882
3	Linear Regression	19588	27460	0.8635



18. Input Features

The application exposes the following user-facing input fields, grouped for clarity. All remaining model features not shown to the user are automatically filled using data-driven default values (median for continuous features, mode for binary/categorical features) computed from the training data.

Group	Fields
Size	Above-ground living area, total basement area, lot area, garage area
Rooms & Quality	Overall quality (1–10), full bathrooms, half bathrooms, bedrooms, kitchen quality
Location & Age	Neighborhood, year built, year sold, has fireplace, has pool
Model Selection	Choice of Gradient Boosting (tuned), Random Forest, Ridge Regression, or Linear Regression

19. Output

Upon clicking “Predict Price,” the application returns:

- A predicted sale price in US dollars.
- A lower and upper estimate range, calculated as the prediction $\pm$ the model's RMSE on the test set — an honest, regression-appropriate substitute for a classification-style prediction probability.
- A feature importance chart (for tree-based models) or a coefficient chart (for linear models), showing which inputs most influenced predictions overall.

20. Key Highlights

- Built a complete, deployed ML application rather than a notebook-only solution, satisfying the core project requirement.
- Applied context-aware missing value handling rather than blind imputation, preserving the true meaning of the data.
- Implemented all five bonus features: confidence range, feature importance, a dedicated data visualization page, a model performance comparison page, and multiple model selection.
- Achieved a strong, realistic R^2 score of 0.9273 after tuning, with a MAE of roughly 7.6% of the average sale price.
- Designed a professional, custom-styled user interface with a cohesive color palette, elegant typography, and a working Light/Dark theme.
- Fully deployed and publicly accessible via Streamlit Community Cloud, connected to a version-controlled GitHub repository.

21. Conclusion

This project successfully demonstrates the complete lifecycle of a machine learning application: from raw, real-world data through cleaning, feature engineering, model training, comparison, tuning, and finally deployment as an interactive, publicly accessible web application. The final tuned Gradient Boosting model achieved strong predictive performance ($R^2 = 0.9273$), and the accompanying Streamlit application fulfills every required and bonus feature outlined in the task brief. Beyond meeting the technical requirements, particular attention was paid to data integrity, honest metric reporting, and professional interface design — reflecting not just technical competence but a production-oriented approach to machine learning system design.

22. GitHub Repository

The complete source code, notebook, saved models, and documentation for this project are available in the public GitHub repository below:

<https://github.com/maryamkhan68/house-price-prediction-task4>

Live deployed application:

<https://house-price-prediction-task4.streamlit.app>